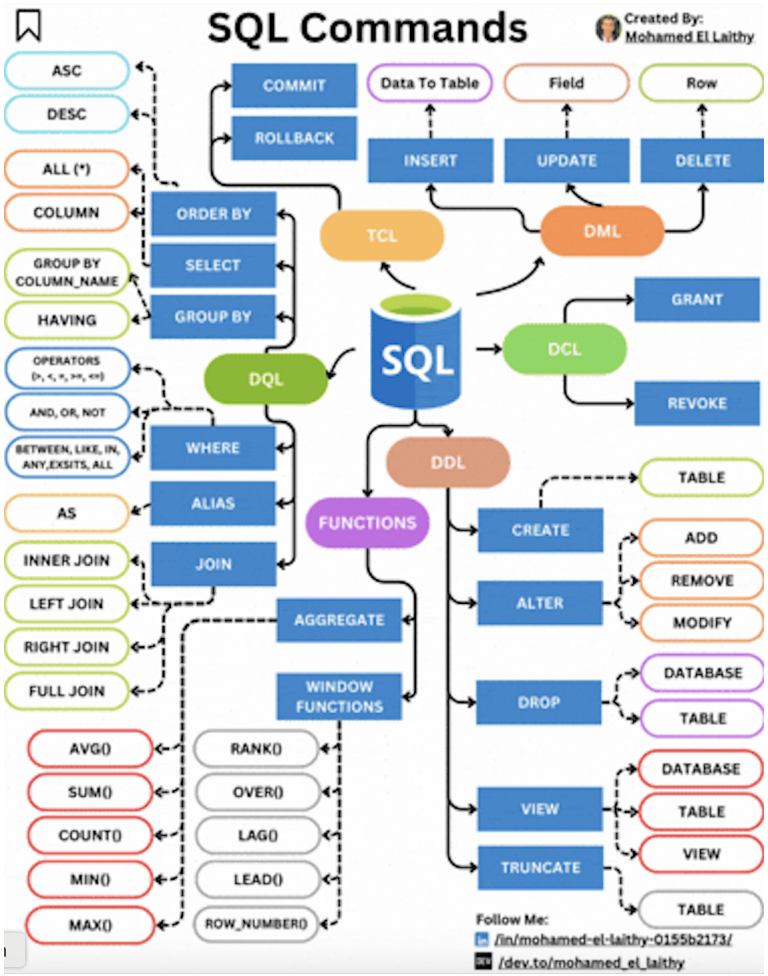
Databases and SQL for Data Science with Python

Matthieu Vanhoecke



Contents

I SQL for Data Manipulation **3**
1 Introduction to Databases . . . 3
What is SQL? . . . 3
What is Data? . . . 4
What is a Database? . . . 4
Relational Database Management Systems (RDBMS) . . . 4
Basic SQL Commands . . . 4
2 SELECT Statement . . . 4
Introduction . . . 4
General Form of `SELECT` . . . 4
Comparison Operators . . . 5
Pattern Matching with `LIKE` . . . 5
Using Ranges with `BETWEEN` . . . 5

Using Sets of Values with `IN` . . . 6
Combining Conditions with `AND` and `OR` . . . 6
Sorting Results with `ORDER BY` . . . 7
3 COUNT, DISTINCT, LIMIT, OFFSET . . . 7
COUNT . . . 7
DISTINCT . . . 8
LIMIT . . . 8
OFFSET . . . 8
4 Grouping and Aggregating SELECT Results . . . 9
Introduction . . . 9
Eliminating Duplicates with `DISTINCT` . . . 9
Grouping Data with `GROUP BY` . . . 9
Renaming Columns with `AS` . . . 10
Filtering Groups with `HAVING` . . . 10
5 INSERT Statement . . . 10
Introduction . . . 10
General Form of `INSERT` . . . 10
6 UPDATE and DELETE Statements . . . 11
Introduction . . . 11
General Form of `UPDATE` . . . 11
General Form of `DELETE` . . . 11

II Functions, Multiple Tables and Sub-queries **13**
1 Introduction . . . 13
2 Aggregate Functions . . . 13
3 Scalar Functions . . . 14
4 String Functions . . . 14
5 Date and Time Functions . . . 15
Introduction . . . 15
Extracting Date and Time Components . . . 15
Date and Time Arithmetic . . . 15
6 Subqueries (Nested SELECT Statements) . . . 16
Subquery in the WHERE Clause . . . 17
Subquery in the SELECT List (Column Expression) . . . 17
Subquery in the FROM Clause (Derived Table) . . . 17
7 Accessing Multiple Tables: Using Subqueries with Multiple Tables . . . 17
8 Accessing Multiple Tables: Implicit Joins (Multiple Tables in FROM Clause) . . . 18
Notes . . . 19
9 Accessing Multiple Tables: JOIN Operations . . . 19
INNER JOIN . . . 19
CROSS JOIN . . . 19
LEFT (OUTER) JOIN . . . 19
RIGHT (OUTER) JOIN . . . 19
FULL (OUTER) JOIN . . . 20
SELF JOIN . . . 20
NATURAL JOIN . . . 20
Summary of JOIN Types . . . 20

III Types of SQL Statements and DDL **21**
1 Database Models and Design . . . 21
The Relational Model . . . 21
Entity-Relationship (ER) Model . . . 21
Mapping Entities to Tables . . . 21
Primary and Foreign Keys . . . 22
2 Relational Model Constraints . . . 22
3 Categories of SQL Statements . . . 22
Data Definition Language (DDL) . . . 22
Data Manipulation Language (DML) . . . 22

4 The CREATE TABLE Statement . . . 22
General Syntax . . . 23
Data Types and Constraints . . . 23
Example: Creating an Author Table . . . 23
5 Modifying and Deleting Tables (ALTER, DROP, TRUNCATE) . . . 24
ALTER TABLE Statement . . . 24
DROP TABLE Statement . . . 25
TRUNCATE TABLE Statement . . . 25

IV Accessing DataBases Using Python **26**
1 Why Use Python with Databases . . . 26
Python Database API (DB-API) . . . 26
Basic Workflow of Database Access . . . 26
Using Jupyter Notebooks . . . 26
2 SQL APIs and Proprietary Database APIs . . . 27
3 Writing Code Using the Python DB-API . . . 27
Overview of the Python DB-API . . . 27
Common DB-API Implementations . . . 27
Core DB-API Concepts . . . 27
Database Cursors . . . 28
Typical DB-API Workflow . . . 28
DB-API Usage Examples . . . 28
4 Accessing Databases Using SQL Magic . . . 29
Magic Statements in Jupyter Notebooks . . . 29
Commonly Used Line Magics . . . 30
Cell Magics and Multi-language Support . . . 30
Using SQL Magic in Jupyter Notebooks . . . 30
SQL Magic Usage Examples . . . 30
5 Analyzing Data with Python . . . 31
6 Loading and Storing the Dataset . . . 31
Retrieving Data from the Database . . . 31
Exploratory Data Analysis with Pandas . . . 32
Analyzing Sodium Content . . . 32
Visualizing Sodium with Seaborn . . . 32
Relationship Between Protein and Total Fat . . . 32
Box Plot Analysis for Sugar Content . . . 33
Summary . . . 33

I SQL for Data Manipulation

Databases store valuable information, but data is only useful if we can efficiently retrieve and manipulate it. SQL (Structured Query Language) is the standard tool for working with relational databases. This section introduces the basics of SQL, how to query data, and some practical functions that help us work with tables.

1 Introduction to Databases

What is SQL?

SQL is a language used to query and manipulate relational databases. With SQL you can:

- Retrieve data
- Insert new data
- Update existing data
- Delete data

What is Data?

Data is a collection of facts in the form of text, numbers, or images. Businesses and applications collect data constantly, such as customer information, transactions, or sensor readings. Efficient storage and retrieval are crucial.

What is a Database?

A database is an organized repository of data. Relational databases store data in **tables** (rows and columns), where:

- Columns describe properties of items (e.g., `FirstName`, `Email`)
- Rows represent records or items (e.g., employees, books)

Relational Database Management Systems (RDBMS)

An RDBMS is a software system that manages relational databases. It handles:

- Storing and organizing data
- Controlling access
- Querying efficiently

Examples: MySQL, Oracle Database, DB2 Warehouse, DB2 on Cloud.

Basic SQL Commands

Five fundamental SQL commands are enough for most database tasks:

- `CREATE TABLE` – create a new table
- `INSERT` – insert data into a table
- `SELECT` – retrieve data from a table
- `UPDATE` – update existing data
- `DELETE` – remove data from a table

2 SELECT Statement

Introduction

The `SELECT` statement is used to read data from a database. The result of a query is called a **result set**. It is part of the Data Manipulation Language (DML), which focuses on retrieving and modifying data.

General Form of `SELECT`

Formula

```
SELECT <column_name(s)> FROM <table_name> [WHERE <condition>];
```

- `<column_name(s)>` – columns to retrieve (use * for all)
- `<table_name>` – table to query
- `[WHERE <condition>]` – optional filter for rows

Examples

- Retrieve all columns from the table:

  ```
  SELECT * FROM book;
  ```

  All rows and columns of the `book` table are returned.

- Retrieve specific columns (`book_id` and `title`):

  ```
  SELECT book_id, title FROM book;
  ```

  Only the columns `book_id` and `title` are returned.

- Retrieve rows with a condition (book_id = 'B1'):

  ```
  SELECT book_id, title
  FROM book
  WHERE book_id = 'B1';
  ```

  Only rows satisfying the condition are returned.

Comparison Operators

Common operators in the `WHERE` clause:

- `=` equal to
- `<>` or `!=` not equal to
- `>` greater than
- `<` less than
- `>=` greater than or equal to
- `<=` less than or equal to

Pattern Matching with `LIKE`

When the exact value of a string is unknown, SQL provides the `LIKE` predicate to search for patterns within text columns. The `LIKE` predicate is used in the `WHERE` clause together with wildcard characters.

Formula

```
SELECT <column_name(s)> FROM <table_name> WHERE <column> LIKE <pattern>;
```

- `%` – represents zero, one, or multiple characters

Example Retrieve authors whose first name starts with the letter `R`:

```
SELECT first_name
FROM author
WHERE first_name LIKE 'R%';
```

All rows where the `first_name` begins with `R` are returned.

Using Ranges with `BETWEEN`

The `BETWEEN ... AND` operator is used to filter values within a specific range. The boundary values are included in the result.

Formula

```
SELECT <column_name(s)> FROM <table_name> WHERE <column> BETWEEN <value1> AND <value2>;
```

Example Retrieve books with a number of pages between 290 and 300:

```
SELECT title
FROM book
WHERE pages BETWEEN 290 AND 300;
```

Only rows with values within the specified range are returned.

Using Sets of Values with IN

When filtering data based on a list of specific values, the IN operator can be used instead of multiple conditions.

Formula

```
SELECT <column_name(s)> FROM <table_name> WHERE <column> IN (<value1>, <value2>, ...);
```

Example Retrieve authors from specific countries:

```
SELECT author_name
FROM author
WHERE country IN ('Australia', 'Brazil', 'Canada');
```

Only rows matching one of the listed values are returned.

Combining Conditions with AND and OR

Multiple conditions can be combined in the WHERE clause using logical operators. Logical operators allow more precise control over which rows are included in the result set.

Formula

```
SELECT <column_name(s)> FROM <table_name> WHERE <condition1> AND|OR <condition2>;
```

- AND – returns rows only if **all** conditions are true
- OR – returns rows if **at least one** condition is true

Examples

- Retrieve books with more than 290 pages and less than 300 pages:

  ```
  SELECT title
  FROM book
  WHERE pages > 290 AND pages < 300;
  ```

  Only rows satisfying both conditions are returned.

- Retrieve books with fewer than 100 pages or more than 500 pages:

  ```
  SELECT title
  FROM book
  WHERE pages < 100 OR pages > 500;
  ```

  Rows satisfying either condition are returned.

Sorting Results with ORDER BY

By default, the result set returned by a SELECT statement has no guaranteed order. To display query results in a specific order, SQL provides the ORDER BY clause.

Formula

```
SELECT <column_name(s)> FROM <table_name> ORDER BY <column_name | column_position> [ASC |
DESC];
```

- ORDER BY – specifies the column used for sorting
- ASC – ascending order (default)
- DESC – descending order

Examples

- Retrieve all book titles in alphabetical order:

  ```
  SELECT title
  FROM book
  ORDER BY title;
  ```

  The result set is sorted in ascending (A–Z) order by default.

- Retrieve book titles in reverse alphabetical order:

  ```
  SELECT title
  FROM book
  ORDER BY title DESC;
  ```

  The result set is sorted from Z to A.

- Sort books by number of pages (ascending):

  ```
  SELECT title, pages
  FROM book
  ORDER BY pages;
  ```

  Books are ordered by increasing number of pages.

- Sort using column position instead of column name:

  ```
  SELECT title, pages
  FROM book
  ORDER BY 2;
  ```

  The result set is sorted by the second column in the SELECT list, which is pages.

  Note: When values in the sorted column are identical, SQL continues sorting using subsequent characters or values until a difference is found.

3 COUNT, DISTINCT, LIMIT, OFFSET

COUNT

COUNT counts the number of rows matching a query.

Formula

```
SELECT COUNT(<column_name>) FROM <table_name> [WHERE <condition>];
```

Examples:

- Count all rows: `SELECT COUNT(*) FROM MEDALS;`
- Count rows matching a condition:

```
SELECT COUNT(COUNTRY)
FROM MEDALS
WHERE COUNTRY = 'CANADA';
```

DISTINCT

`DISTINCT` removes duplicate values from the result set.

Formula

```
SELECT DISTINCT <column_name> FROM <table_name> [WHERE <condition>];
```

Examples:

- Retrieve unique countries: `SELECT DISTINCT COUNTRY FROM MEDALS;`
- Unique countries with gold medals:

```
SELECT DISTINCT COUNTRY
FROM MEDALS
WHERE MEDALTYPE = 'GOLD';
```

LIMIT

`LIMIT` restricts the number of rows returned. LIMIT without ORDER BY always returns the **first rows**, never the last ones.

Formula

```
SELECT <column_name(s)> FROM <table_name> [WHERE <condition>] LIMIT <number>;
```

Examples:

- First 10 rows: `SELECT * FROM MEDALS LIMIT 10;`
- First 5 rows for 2018:

```
SELECT * FROM MEDALS
WHERE YEAR = 2018
LIMIT 5;
```

OFFSET

The `OFFSET` keyword allows you to skip a certain number of rows before starting to return results. It is often used together with `LIMIT` for **pagination** of query results.

Formula

```
SELECT <column_name(s)> FROM <table_name> [WHERE <condition>] LIMIT <number> OFFSET
<number>;
```

Examples:

- Skip the first 5 rows and retrieve the next 10 rows:

```
SELECT *
FROM MEDALS
LIMIT 10 OFFSET 5;
```

- Skip the first 20 rows for the year 2018 and retrieve the next 5 rows:

```
SELECT *
FROM MEDALS
WHERE YEAR = 2018
LIMIT 5 OFFSET 20;
```

4 Grouping and Aggregating SELECT Results

Introduction

Sometimes a `SELECT` query returns duplicate values or you want to summarize data. SQL provides keywords to **eliminate duplicates, group data, and perform aggregations**. The most important tools are:

- `DISTINCT` – removes duplicate rows
- `GROUP BY` – groups rows sharing common values
- `HAVING` – filters groups based on conditions

Eliminating Duplicates with `DISTINCT`

The `DISTINCT` keyword reduces a result set to **unique values only**.

Formula

```
SELECT DISTINCT <column_name(s)> FROM <table_name> [ORDER BY <column>];
```

Example Retrieve a list of unique countries for authors:

```
SELECT DISTINCT country
FROM author
ORDER BY 1;
```

Only unique countries are returned, sorted alphabetically.

Grouping Data with `GROUP BY`

The `GROUP BY` clause groups rows that have the same values in one or more columns. It is often used with **aggregate functions** like `COUNT`, `SUM`, or `AVG`.

Formula

```
SELECT <column_name>, AGGREGATE_FUNCTION(<column>) FROM <table_name> GROUP BY
<column_name>;
```

Example Count the number of authors from each country:

```
SELECT country, COUNT(*)
FROM author
GROUP BY country;
```

The result set shows each country and the number of authors from that country. By default, the second column is named after the aggregate function (e.g., "COUNT(*)").

Renaming Columns with AS

Use AS to assign a **meaningful name** to a derived column:

Formula

```
SELECT <column>, AGGREGATE_FUNCTION(<column>) AS <alias> FROM <table> GROUP BY <column>;
```

Example Rename the count column to AuthorCount:

```
SELECT country, COUNT(*) AS AuthorCount
FROM author
GROUP BY country;
```

Filtering Groups with HAVING

The HAVING clause sets conditions **on groups**, unlike WHERE, which filters individual rows.

Formula

```
SELECT <column>, AGGREGATE_FUNCTION(<column>) AS <alias> FROM <table> GROUP BY <column>
HAVING <aggregate_condition>;
```

Example Only show countries with more than 4 authors:

```
SELECT country, COUNT(*) AS AuthorCount
FROM author
GROUP BY country
HAVING COUNT(*) > 4;
```

Result: Only countries with five or more authors are returned (e.g., China, India).

5 INSERT Statement

Introduction

The INSERT statement is used to add new rows to a relational database table. It is part of the **Data Manipulation Language (DML)**, which allows you to read and modify data. After a table is created, it needs to be populated with data. There are **two main ways** to add rows:

- Insert **one row at a time**
- Insert **multiple rows at once**

General Form of INSERT

Formula

```
INSERT INTO <table_name> (<column_name(s)>) VALUES (<value(s)>);
```

- <table_name> – the table you want to insert data into
- <column_name(s)> – list of columns being populated
- <value(s)> – values for each column (order must match column list)

Examples

- Insert a single row into the `author` table:

```
INSERT INTO author (author_id, last_name, first_name, email, city, country)
VALUES ('A1', 'Chong', 'Raul', 'RFC@IBM.com', 'Toronto', 'CA');
```

- Insert multiple rows at once:

```
INSERT INTO author (author_id, last_name, first_name, email, city, country)
VALUES
('A1', 'Chong', 'Raul', 'RFC@IBM.com', 'Toronto', 'CA'),
('A2', 'Ahuja', 'Rav', 'RA@IBM.com', 'Vancouver', 'CA');
```

Notes: - The number of values in each row must match the number of columns specified - Multiple rows are separated by commas inside the `VALUES` clause

6 UPDATE and DELETE Statements

Introduction

After a table is created and populated with data, you may want to modify or remove rows.
- The `UPDATE` statement is used to alter existing data.
- The `DELETE` statement is used to remove rows from a table.
Both are part of the Data Manipulation Language (DML). It is important to always use the WHERE clause; without it, all rows in the table may be affected.

General Form of `UPDATE`

Formula

```
UPDATE <table_name> SET <column_name> = <value> [, ...]  [WHERE <condition>];
```

Parameters:

- `<table_name>` – the table to update
- `<column_name> = <value>` – column(s) and new values
- `[WHERE <condition>]` – optional filter to update specific rows

General Form of `DELETE`

Formula

```
DELETE FROM <table_name> [WHERE <condition>];
```

Parameters:

- `<table_name>` – the table to delete rows from
- `[WHERE <condition>]` – optional filter to delete specific rows

Examples

- Update the first and last name of the author with `AUTHOR_ID = 'A2'`:

  ```
  UPDATE author
  SET last_name = 'Katta', first_name = 'Lakshmi'
  WHERE author_id = 'A2';
  ```

- Delete authors with `AUTHOR_ID = 'A2'` or `'A3'`:

  ```
  DELETE FROM author
  WHERE author_id IN ('A2', 'A3');
  ```

  Note: Always include the **WHERE clause**. Without it: - `UPDATE` will change all rows - `DELETE` will remove all rows

II Functions, Multiple Tables and Sub-queries

1 Introduction

SQL provides built-in functions that allow operations to be performed **directly in the database**, reducing the amount of data transferred to applications and improving efficiency.

Functions can operate on:

- Entire columns (aggregate functions)
- Individual values (scalar functions)
- Strings (string functions)

Using functions inside SQL queries is more efficient than retrieving data first and performing operations in an application.

2 Aggregate Functions

Aggregate functions take a collection of values (typically in a column) and return a **single value**.

Formula

```
SELECT AGGREGATE_FUNCTION(<column>) [AS <alias>] FROM <table_name> [WHERE <condition>];
```

Common Aggregate Functions

- `SUM(<column>)` – returns the sum of all values
- `MIN(<column>)` – returns the minimum value
- `MAX(<column>)` – returns the maximum value
- `AVG(<column>)` – returns the average value
- `COUNT(<column>)` – returns the number of rows

Examples Using the `PETRESCUE` table with columns `ID, ANIMAL, QUANTITY, COST, RESCUE_DATE`:

- Total cost of all rescues:

```
SELECT SUM(COST) AS SUM_OF_COST
FROM PETRESCUE;
```

- Minimum ID for dog rescues:

```
SELECT MIN(ID)
FROM PETRESCUE
WHERE ANIMAL = 'Dog';
```

- Maximum quantity in a single rescue:

```
SELECT MAX(QUANTITY)
FROM PETRESCUE;
```

- Average cost per dog (cost divided by quantity):

```
SELECT AVG(COST / QUANTITY)
FROM PETRESCUE
WHERE ANIMAL = 'Dog';
```

3 Scalar Functions

Scalar functions perform operations on **individual values** in a column.

Formula

```
SELECT SCALAR_FUNCTION(<column>) FROM <table_name> [WHERE <condition>];
```

Numeric Functions

- `ROUND(column, n)` – rounds a numeric value to n decimal places
- `CEIL(column)` – returns the smallest integer greater than or equal to the value
- `FLOOR(column)` – returns the largest integer less than or equal to the value
- `ABS(column)` – returns the absolute value

Examples

- Round cost to nearest integer:

  ```
  SELECT ROUND(COST)
  FROM PETRESCUE;
  ```

- Retrieve lowercase animal names for cats:

  ```
  SELECT *
  FROM PETRESCUE
  WHERE LOWER(ANIMAL) = 'cat';
  ```

4 String Functions

String functions operate on character or text columns (`CHAR, VARCHAR`).

Formula

```
SELECT STRING_FUNCTION(<column>) FROM <table_name> [WHERE <condition>];
```

Common String Functions

- `LENGTH(<column>)` – returns the length of the string
- `UPPER(<column>)` – converts string to uppercase
- `LOWER(<column>)` – converts string to lowercase

Examples

- Length of each animal name:

  ```
  SELECT LENGTH(ANIMAL)
  FROM PETRESCUE;
  ```

- Animal names in uppercase:

  ```
  SELECT UPPER(ANIMAL)
  FROM PETRESCUE;
  ```

- Unique uppercase animal names:

  ```
  SELECT DISTINCT UPPER(ANIMAL)
  FROM PETRESCUE;
  ```

5 Date and Time Functions

Introduction

SQL provides special data types and functions to work with **dates and times**. Most databases support the following types:

- `DATE` – 8 digits, format `YYYYMMDD`
- `TIME` – 6 digits, format `HHMMSS`
- `TIMESTAMP` – 20 digits, format `YYYYMMDDHHMMSSZZZZZZ` (`ZZZZZZ` = microseconds)

Date and time functions can be used to extract specific components, perform calculations, and filter data.

Extracting Date and Time Components

Functions exist to extract various parts of a date or time:

- `DAY()` – day of the month
- `MONTH()` – month of the year
- `YEAR()` – year
- `WEEK()` – week number
- `HOUR()`, `MINUTE()`, `SECOND()` – time components
- `DAYOFWEEK()` – day of the week
- `DAYOFYEAR()` – day of the year

Formula

```
SELECT FUNCTION(<column>) FROM <table_name> [WHERE <condition>];
```

Examples

- Extract day from rescue dates for cats:

```
SELECT DAY(rescue_date)
FROM PETRESCUE
WHERE ANIMAL = 'Cat';
```

- Count rescues in the month of May (month = 5):

```
SELECT COUNT(*)
FROM PETRESCUE
WHERE MONTH(rescue_date) = 5;
```

Date and Time Arithmetic

SQL allows arithmetic with dates and times to calculate intervals or durations.

Formula

```
SELECT <column> + INTERVAL <number> <unit> FROM <table_name>;
```

Common Functions

- `DATE_ADD(date, INTERVAL n unit)` – adds time to a date
- `DATE_SUB(date, INTERVAL n unit)` – subtracts time from a date
- `DATEDIFF(date1, date2)` – difference in days between two dates
- `TIMEDIFF(time1, time2)` – difference between two times
- `NOW()` – current date and time
- `CURDATE()` – current date only
- `CURTIME()` – current time only
- `EXTRACT(unit FROM date)` – extracts a part of the date/time (e.g., year, month, day)
- `FROM_DAYS(n)` – converts a number of days into a date

Examples

- Date three days after each rescue:

```
SELECT DATE_ADD(rescue_date, INTERVAL 3 DAY)
FROM PETRESCUE;
```

- Date two weeks before each rescue:

```
SELECT DATE_SUB(rescue_date, INTERVAL 2 WEEK)
FROM PETRESCUE;
```

- Days elapsed since each rescue until today:

```
SELECT DATEDIFF(CURDATE(), rescue_date)
FROM PETRESCUE;
```

- Time difference between two columns:

```
SELECT TIMEDIFF(end_time, start_time)
FROM PETRESCUE;
```

- Extract month and year from rescue date:

```
SELECT EXTRACT(MONTH FROM rescue_date), EXTRACT(YEAR FROM rescue_date)
FROM PETRESCUE;
```

Notes

- Date and time functions can also be used in the `WHERE` clause.
- Arithmetic can be performed using addition/subtraction of intervals or timestamps.
- These functions help filter, calculate, and analyze data based on time periods.

6 Subqueries (Nested SELECT Statements)

A **subquery** (also called a **nested SELECT**) is a query placed inside another SQL query. Subqueries are enclosed in parentheses and allow more powerful data retrieval. A subquery can appear in:

- the `WHERE` clause
- the `SELECT` list (column expressions)
- the `FROM` clause (derived tables)

Subquery in the WHERE Clause

Formula

```
SELECT <column_name(s)> FROM <table_name> WHERE <column> <operator> (SELECT
<aggregate_function> FROM <table_name>);
```

Example : Retrieve employees who earn more than the average salary:

```
SELECT employee_id, first_name, last_name, salary
FROM employees
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
);
```

Subquery in the SELECT List (Column Expression)

Formula

```
SELECT <column>, (SELECT <aggregate_function> FROM <table>) AS <alias> FROM <table_name>;
```

Example : Display each employee's salary along with the average salary:

```
SELECT employee_id, salary,
       (SELECT AVG(salary) FROM employees) AS avg_salary
FROM employees;
```

Subquery in the FROM Clause (Derived Table)

Formula

```
SELECT <column_name(s)> FROM (SELECT <column_name(s)> FROM <table_name>) AS <alias>;
```

Example : Create a derived table with non-sensitive employee data:

```
SELECT *
FROM (
    SELECT employee_id, first_name, last_name, department_id
    FROM employees
) AS employee_info;
```

Notes on Subqueries :

- Subqueries must be enclosed in parentheses
- Aggregate functions are often used inside subqueries
- Subqueries in the `FROM` clause must have an alias

7 Accessing Multiple Tables: Using Subqueries with Multiple Tables

A subquery can be used to filter rows in one table based on data stored in another table.

Formula

```
SELECT <column_name(s)> FROM <table1> WHERE <column> IN (SELECT <column> FROM <table2>);
```

Examples

- Retrieve employees whose department exists in the `departments` table:

```
SELECT *
FROM employees
WHERE dep_id IN (
    SELECT dep_id
    FROM departments
);
```

- Retrieve employees working in a specific location:

```
SELECT *
FROM employees
WHERE dep_id IN (
    SELECT dep_id
    FROM departments
    WHERE loc_id = 'L0002'
);
```

- Retrieve department details for employees earning more than 70,000:

```
SELECT dep_id, dep_name
FROM departments
WHERE dep_id IN (
    SELECT dep_id
    FROM employees
    WHERE salary > 70000
);
```

8 Accessing Multiple Tables: Implicit Joins (Multiple Tables in FROM Clause)

Multiple tables can be accessed by listing them in the `FROM` clause. This creates a join between tables without explicitly using a JOIN keyword.

Formula

```
SELECT <column_name(s)> FROM <table1>, <table2> [WHERE <join_condition>];
```

Examples

- Cartesian product (no condition specified), **Cross Join**:

```
SELECT *
FROM employees, departments;
```

- Join employees with their departments using a condition, **Inner Join**:

```
SELECT *
FROM employees, departments
WHERE employees.dep_id = departments.dep_id;
```

- Using table **aliases** for readability:

```
SELECT *
FROM employees E, departments D
WHERE E.dep_id = D.dep_id;
```

- Retrieve employee ID and department name:

```
SELECT E.emp_id, D.dep_name
FROM employees E, departments D
WHERE E.dep_id = D.dep_id;
```

The aliases notation and the notations form the two last examples are very usefull

Notes

- Always qualify column names when multiple tables are used
- Table aliases make queries shorter and easier to read
- Implicit joins are logically equivalent to INNER JOINs when a matching condition is specified

9 Accessing Multiple Tables: JOIN Operations

SQL provides explicit `JOIN` operators to combine rows from two or more tables based on related columns. Using JOINs improves readability and makes query intent clearer compared to implicit joins.

Formula

```
SELECT <column_name(s)> FROM <table1> <JOIN_TYPE> <table2> ON <join_condition>;
```

INNER JOIN

Returns only rows that have matching values in both tables.

```
SELECT E.emp_id, D.dep_name
FROM employees E
INNER JOIN departments D
ON E.dep_id = D.dep_id;
```

Result: Rows with matching department IDs in both tables.

CROSS JOIN

Returns the Cartesian product of both tables (every row from the first table combined with every row from the second table).

```
SELECT *
FROM employees
CROSS JOIN departments;
```

Result: Number of rows = rows in employees $\times$ rows in departments.

LEFT (OUTER) JOIN

Returns all rows from the left table and matching rows from the right table. Non-matching rows from the right table return `NULL` values.

```
SELECT E.emp_id, D.dep_name
FROM employees E
LEFT JOIN departments D
ON E.dep_id = D.dep_id;
```

RIGHT (OUTER) JOIN

Returns all rows from the right table and matching rows from the left table. Non-matching rows from the left table return `NULL` values.

```
SELECT E.emp_id, D.dep_name
FROM employees E
RIGHT JOIN departments D
ON E.dep_id = D.dep_id;
```

FULL (OUTER) JOIN

Returns all rows from both tables. Rows without matches on either side contain `NULL` values.

```
SELECT E.emp_id, D.dep_name
FROM employees E
FULL JOIN departments D
ON E.dep_id = D.dep_id;
```

SELF JOIN

A table is joined with itself. Useful for hierarchical or recursive relationships (e.g., employees and managers).

```
SELECT E.emp_id, M.emp_id AS manager_id
FROM employees E
LEFT JOIN employees M
ON E.manager_id = M.emp_id;
```

NATURAL JOIN

Automatically joins tables based on columns with the same name and compatible data types. No `ON` clause is required.

```
SELECT *
FROM employees
NATURAL JOIN departments;
```

> **Note:** `NATURAL JOIN` should be used with caution, as it depends on column naming conventions.

Summary of JOIN Types

- `INNER JOIN` – matching rows only
- `CROSS JOIN` – all combinations of rows
- `LEFT JOIN` – all rows from left table
- `RIGHT JOIN` – all rows from right table
- `FULL JOIN` – all rows from both tables
- `SELF JOIN` – table joined with itself
- `NATURAL JOIN` – automatic join on same-named columns

III Types of SQL Statements and DDL

SQL statements interact with **Entities** (tables), **Attributes** (columns), and their **Tuples** (rows). These statements generally fall into two main categories: Data Definition Language (DDL) and Data Manipulation Language (DML).

1 Database Models and Design

Before writing SQL code, databases are designed using models. The most common are the Relational Model and the Entity-Relationship (ER) Model.

The Relational Model

The relational model is the most widely used data model because it offers **Data Independence**:

- **Logical Data Independence:** Changing the schema (structure) does not affect the data.
- **Physical Data Independence:** Changing how data is stored physically does not affect the logical structure.

Entity-Relationship (ER) Model

The ER model is a tool used to design relational databases. It visualizes data as a collection of **Entities and Attributes**.

- **Entity (Table):** An object that exists independently (Person, Place, Thing).
- **Attribute (Column):** Data elements that characterize the entity.

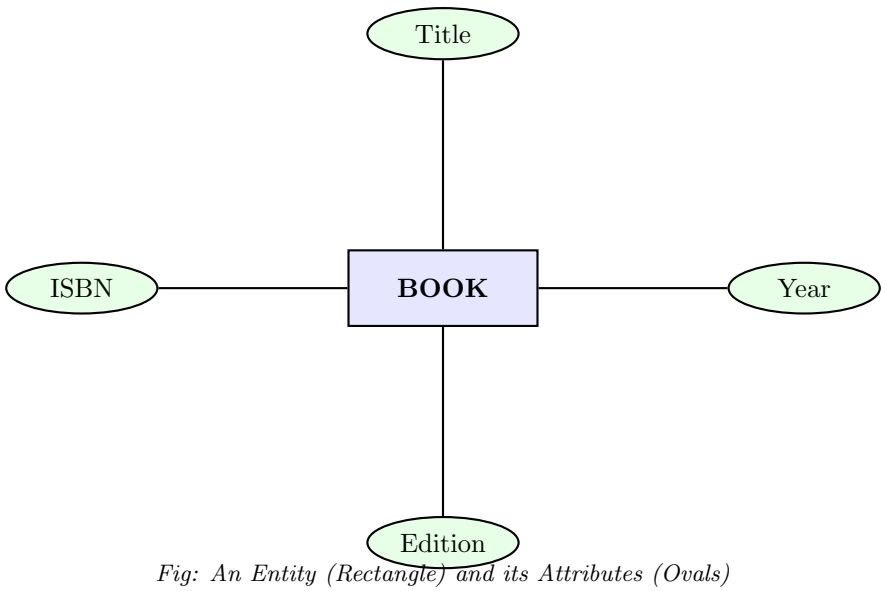


Fig: An Entity (Rectangle) and its Attributes (Ovals)

Mapping Entities to Tables

Once the design is complete, we map the ER diagram to a Relational Database:

ER Model	Map To	Relational Database
Entity	→	**Table**
Attribute	→	**Column**
Instance	→	**Row (Tuple)**

During this phase, we choose specific data types for columns (e.g., using `VARCHAR` for variable titles or `CHAR` for fixed-length codes like ISBNs which may contain dashes).

Primary and Foreign Keys

Keys are fundamental to the relational model for identity and relationships.

- **Primary Key (PK):** A column (or set of columns) that **uniquely identifies** each row in a table. It prevents duplicate data.
- **Foreign Key (FK):** A primary key from one table that appears in another table. It creates a **link** (relationship) between the two tables.

2 Relational Model Constraints

To maintain data integrity in a relational database, the relational model defines three essential types of constraints. These constraints ensure that data remains accurate, consistent, and logically connected.

- **Entity Integrity:** Ensures that each table has a **Primary Key** that uniquely identifies every row. Primary key values must be **unique** and **NOT NULL**, preventing duplicate or unidentified records.
- **Referential Integrity:** Ensures that relationships between tables remain valid by enforcing **Foreign Key** constraints. A foreign key value must reference an existing primary key in another table, preventing orphan or invalid references.
- **Domain Integrity:** Ensures that all values stored in a column are valid according to defined rules such as: data type, format, allowed range, and nullability. This is enforced using constraints like `CHECK` and `NOT NULL`.

Together, these constraints preserve the reliability and correctness of data in a relational database.

3 Categories of SQL Statements

Data Definition Language (DDL)

DDL statements are used to define, change, or drop database objects such as tables. They manage the structure of the database.

Common DDL statements include:

- `CREATE`: Used for creating tables and defining their columns.
- `ALTER`: Used for modifying table structures (e.g., adding/dropping columns, changing data types).
- `TRUNCATE`: Used for deleting all data in a table but keeping the table structure itself.
- `DROP`: Used for deleting tables entirely (structure and data).

Data Manipulation Language (DML)

DML statements are used to read and modify data within existing tables. These operations are often referred to as **CRUD** (Create, Read, Update, Delete).

Common DML statements include:

- `INSERT` (Create): Inserts a row or several rows of data.
- `SELECT` (Read): Reads or selects rows from a table.
- `UPDATE` (Update): Edits existing rows in a table.
- `DELETE` (Delete): Removes rows of data from a table.

4 The CREATE TABLE Statement

The `CREATE TABLE` statement is the most common DDL command. It allows you to define an entity (table name) and its attributes (columns) with specific data types and constraints.

General Syntax

Formula

```
CREATE TABLE <table_name> (
   <column_1> <datatype> [optional_constraints],
   <column_2> <datatype> [optional_constraints],
   ...
);
```

Data Types and Constraints

When defining columns, you must specify the data type:

- `CHAR(n)`: A character string of fixed length n.
- `VARCHAR(n)`: A character string of variable length (up to n characters).
- `INTEGER` (or `INT`): Stores whole numbers (e.g., 1, 50, -10).
- `DECIMAL(p,s)`: Stores exact numeric values with fixed precision p (total digits) and scale s (decimal digits), often used for currency.
- `FLOAT` / `DOUBLE`: Stores approximate numeric values (floating-point numbers) for scientific calculations.
- `DATE`: Stores a date value (format: YYYY-MM-DD).
- `TIME`: Stores a time value (format: HH:MM:SS).
- `TIMESTAMP`: Stores both date and time (format: YYYY-MM-DD HH:MM:SS).
- `BOOLEAN`: Stores boolean values (`TRUE` or `FALSE`).

You can also apply constraints to ensure data integrity:

- `PRIMARY KEY`: Uniquely identifies each tuple (row). No duplicate values allowed.
- `NOT NULL`: Ensures the column cannot be empty (must contain a value).
- `UNIQUE`: Ensures all values in a column are different.
- `FOREIGN KEY`: Ensures referential integrity by linking to a Primary Key in another table.

Example: Creating an Author Table

In a Library database, we might create a table for the `AUTHOR` entity.

- **Table Name:** author
- **Columns:** author_id, lastname, firstname, email, city, country
- **Constraints:** author_id is the Primary Key; names cannot be null.

```
CREATE TABLE author (
    author_id CHAR(2) PRIMARY KEY NOT NULL,
    lastname VARCHAR(15) NOT NULL,
    firstname VARCHAR(15) NOT NULL,
    email VARCHAR(40),
    city VARCHAR(15),
    country CHAR(2)
);
```

Executing this statement creates the structure in the database, ready to receive data via DML statements.

5 Modifying and Deleting Tables (ALTER, DROP, TRUNCATE)

While `CREATE` defines the initial structure, requirements often change. SQL provides specific DDL statements to modify existing structures or remove them entirely.

ALTER TABLE Statement

The `ALTER TABLE` statement is used to change the structure of an existing table. Unlike `CREATE TABLE`, it does not use parentheses to enclose parameters.

Formula

```
ALTER TABLE <table_name> <action> <column_name> [options];
```

Common actions include:

- **ADD COLUMN**: Adds a new attribute to the table.

  ```
  ALTER TABLE author ADD COLUMN telephone_number BIGINT;
  ```

- **MODIFY / ALTER COLUMN**: Changes the data type or constraints of an existing column.

  ```
  ALTER TABLE author MODIFY telephone_number CHAR(20);
  ```

- **DROP COLUMN**: Removes a column from the table structure.

  ```
  ALTER TABLE author DROP COLUMN telephone_number;
  ```

- **RENAME COLUMN**: Changes the name of a column.

  ```
  ALTER TABLE author RENAME COLUMN lastname TO last_name;
  ```

- **ADD/DROP CONSTRAINT**: Removes an existing constraint.

  ```
  ALTER TABLE author DROP CONSTRAINT unique_email;
  ALTER TABLE author ADD CONSTRAINT unique_email UNIQUE (email);
  ```

- **ADD/DROP PRIMARY KEY/FOREIGN KEY**: Removes a key constraint.

  ```
  ALTER TABLE author DROP PRIMARY KEY;
  ALTER TABLE author ADD PRIMARY KEY (author_id);
  ```

- **SET DEFAULT**: Assigns a default value to a column.

  ```
  ALTER TABLE author ALTER COLUMN country SET DEFAULT 'US';
  ```

- **DROP DEFAULT**: Removes the default value of a column.

  ```
  ALTER TABLE author ALTER COLUMN country DROP DEFAULT;
  ```

- **RENAME TABLE**: Changes the table name.

  ```
  ALTER TABLE author RENAME TO writer;
  ```

Warning: Altering a data type with existing data can cause errors if the data is incompatible (e.g., converting a `CHAR` column containing letters into a numeric type).

DROP TABLE Statement

The `DROP TABLE` statement is used to remove a table definition and all its data from the database permanently.

Formula

```
DROP TABLE <table_name>;
```

Example:

```
DROP TABLE author;
```

Effect:

- The entire table definition is deleted.
- All rows (data) are permanently lost.
- Indexes, constraints, and relationships associated with the table are also removed.
- The table no longer exists and must be recreated to be used again.

Important: `DROP TABLE` is irreversible and cannot be rolled back in most database systems.

TRUNCATE TABLE Statement

The `TRUNCATE` statement is used to delete all rows within a table while keeping the table structure (columns/types) intact.

Formula

```
TRUNCATE TABLE <table_name> IMMEDIATE;
```

- **IMMEDIATE**: Specifies that the operation is processed instantly and cannot be undone (cannot be rolled back).

Example:

```
TRUNCATE TABLE author IMMEDIATE;
```

Effect:

- All rows in the table are deleted instantly.
- The table structure (columns, data types, constraints) remains intact.
- Storage space is freed efficiently.
- The operation cannot be rolled back.

Comparison with DELETE:

- `TRUNCATE` is faster than `DELETE FROM author;` because it does not log individual row deletions.
- `DELETE` can use a `WHERE` clause; `TRUNCATE` cannot.

IV Accessing DataBases Using Python

Databases are essential tools for data scientists, and Python provides powerful and flexible ways to interact with them. Using Python, you can connect to databases, create tables, load data, execute SQL queries, and analyze results directly within programming environments such as Jupyter notebooks.

1 Why Use Python with Databases

Python is widely used for database access because of its simplicity and rich ecosystem. Some key advantages include:

- Simple and readable syntax
- Open-source and cross-platform support
- Strong ecosystem for data science and analytics

Popular Python packages often used alongside databases include:

- `NumPy` – numerical computing
- `Pandas` – data manipulation and analysis
- `Matplotlib` – data visualization
- `SciPy` – scientific computing

Python Database API (DB-API)

Python supports relational databases through a standardized interface known as the **Python Database API (DB-API)**. The DB-API defines a set of methods that allow Python programs to:

- Connect to a database
- Execute SQL statements
- Retrieve query results
- Handle errors
- Close database connections

This standardization allows Python code to interact with different database systems in a consistent way.

Basic Workflow of Database Access

A typical Python–database interaction follows these steps:

- Establish a connection to the database using an API call
- Send SQL statements to the DBMS as text strings
- Retrieve results and status information
- Handle any errors
- Close the connection

Using Jupyter Notebooks

Jupyter notebooks are commonly used for database access in data science workflows. They provide an interactive, web-based environment that supports:

- Live code execution
- Equations and LaTeX
- Visualizations (plots, charts, images)
- Explanatory text alongside code

Notebooks can be shared easily via email, GitHub, or cloud platforms, making them ideal for collaboration and learning.

2 SQL APIs and Proprietary Database APIs

An **Application Programming Interface (API)** is a collection of functions that allows applications to interact with a database system. SQL APIs enable applications to send SQL commands to a DBMS and retrieve results.

Different database systems provide their own APIs, including:

- **MySQL C API** – access to MySQL databases
- **psycopg2** – PostgreSQL database access from Python
- **IBM_DB** – Python API for IBM DB2 databases
- **dblib** – SQL Server database access
- **ODBC** – database access on Microsoft Windows
- **OCI** – Oracle Call Interface for Oracle databases
- **JDBC** – Java Database Connectivity for Java applications

These APIs allow applications written in different programming languages to communicate efficiently with relational database management systems.

3 Writing Code Using the Python DB-API

The Python Database API (DB-API) is the standard interface for accessing relational databases using Python. It enables developers to write database applications in a consistent way, regardless of the underlying database system. By learning the DB-API, you can write portable Python programs that work across multiple relational databases.

Overview of the Python DB-API

The DB-API defines a set of standard functions and objects that allow Python programs to communicate with a DBMS. Instead of writing database-specific code for each system, the DB-API allows a single program to work with different databases by simply changing the database driver.

Key advantages of using the DB-API include:

- Easy to understand and implement
- Consistent interface across database modules
- Portable code across different database systems
- Broad database connectivity support in Python

Common DB-API Implementations

Each database system provides its own DB-API–compliant library. Some commonly used Python database libraries include:

- `ibm_db` – connection to IBM DB2 databases
- `mysql-connector-python` – connection to MySQL databases
- `psycopg2` – connection to PostgreSQL databases
- `PyMongo` – connection to MongoDB databases

Core DB-API Concepts

The two main concepts in the Python DB-API are **connection objects** and **cursor objects**.

Connection Objects Connection objects are used to establish and manage a connection to the database. They also control transactions and resource management. Common connection methods include:

- `cursor()` – creates and returns a new cursor object
- `commit()` – commits pending transactions to the database
- `rollback()` – rolls back pending transactions
- `close()` – closes the database connection

Cursor Objects Cursor objects are used to execute SQL queries and retrieve results. They behave similarly to file handles, keeping track of the current position within a result set. Important characteristics of cursors include:

- Used to execute SQL statements
- Used to fetch query results row by row
- Multiple cursors from the same connection share changes
- Cursors from different connections may be isolated depending on transaction support

Database Cursors

A database cursor is a control structure that allows traversal over records in a database result set. Just as a file handle tracks position within a file, a cursor tracks position within query results. Once the required data has been processed, the cursor should be closed to release resources.

Typical DB-API Workflow

A typical Python program using the DB-API follows these steps:

- Import the database module
- Establish a connection using the `connect()` function
- Create a cursor object from the connection
- Execute SQL queries using the cursor
- Fetch and process query results
- Close the cursor and the connection

Properly closing database connections is essential to avoid resource leaks and ensure efficient database usage.

DB-API Usage Examples

The following examples illustrate how the Python DB-API is typically used to connect to a database, execute SQL queries, and retrieve results. The examples assume that a DB-API–compliant library is installed and properly configured.

Example 1: Connecting to a Database

```
import ibm_db

conn = ibm_db.connect(
    "DATABASE=db_name;HOSTNAME=hostname;PORT=50000;PROTOCOL=TCPIP;",
    "username",
    "password"
)
```

This example establishes a connection to a database and returns a connection object.

Example 2: Creating a Cursor and Executing a Query

```
cursor = conn.cursor()

cursor.execute("SELECT * FROM employees")
```

A cursor object is created from the connection and used to execute an SQL query.

Example 3: Fetching Query Results

```
rows = cursor.fetchall()

for row in rows:
    print(row)
```

The `fetchall()` method retrieves all rows from the result set.

Example 4: Executing a Parameterized Query

```
cursor.execute(
    "SELECT first_name, last_name FROM employees WHERE salary > ?",
    (70000,)
)
```

Parameterized queries improve security and help prevent SQL injection attacks.

Example 5: Committing Transactions

```
cursor.execute(
    "UPDATE employees SET salary = salary + 1000 WHERE dep_id = 'D1'"
)
conn.commit()
```

The `commit()` method saves changes made during the transaction.

Example 6: Closing Cursor and Connection

```
cursor.close()
conn.close()
```

Always close cursors and connections to free database resources.

> **Note:** The exact module name and connection parameters depend on the database system being used (e.g., `psycopg2` for PostgreSQL or `mysql.connector` for MySQL).

4 Accessing Databases Using SQL Magic

SQL Magic provides a convenient way to run SQL queries directly inside Jupyter Notebooks without writing explicit Python database connection code. It relies on *magic statements*, which are special commands that modify the behavior of the notebook environment rather than acting as standard Python syntax.

Magic Statements in Jupyter Notebooks

Magic statements are commands specific to Jupyter Notebooks that simplify common tasks in data analysis. They are identified by the \% symbol and are divided into two categories:

- **Line Magics**: Prefixed with a single % and operate on a single line of input.
- **Cell Magics**: Prefixed with %% and operate on the entire cell.

Commonly Used Line Magics

Some frequently used line magic commands include:

- `%pwd` – displays the current working directory
- `%ls` – lists files in the current directory
- `%history` – shows the command history
- `%reset` – clears all user-defined variables
- `%who` – lists variables in the namespace
- `%whos` – provides detailed information about variables
- `%matplotlib inline` – displays plots inline
- `%timeit` – measures execution time of a statement

Multiple line magics can be used in the same cell, provided each appears on its own line.

Cell Magics and Multi-language Support

Cell magics apply to the entire cell and can execute code in languages other than Python:

- `%%timeit` – times execution of the entire cell
- `%%writefile filename` – writes the cell contents to a file
- `%%html` – renders HTML code
- `%%javascript` – executes JavaScript
- `%%bash` – executes Bash commands

Using SQL Magic in Jupyter Notebooks

SQL Magic enables direct interaction with databases using SQL syntax inside notebook cells. Before using it, the required extension must be installed and loaded.

Installation and Setup

```
!pip install ipython-sql
```

```
%load_ext sql
```

Connecting to a Database For example, connecting to an SQLite database:

```
%sql sqlite:///HR.db
```

SQL Magic Usage Examples

Example 1: Displaying Table Contents

```
%sql SELECT * FROM employees;
```

Example 2: Multi-line Query Using Cell Magic

```
%%sql
SELECT emp_id, first_name, last_name
FROM employees
WHERE salary > 70000;
```

Example 3: Aggregation Query

```
%sql SELECT COUNT(*) FROM employees;
```

Example 4: Creating a Table

```
%%sql
CREATE TABLE departments (
    dep_id INTEGER,
    dep_name TEXT
);
```

Example 5: Inserting Data

```
%%sql
INSERT INTO departments VALUES (1, 'HR'), (2, 'IT');
```

5 Analyzing Data with Python

Python provides a powerful ecosystem for data analysis, combining data manipulation, database access, and visualization. In this section, we perform exploratory data analysis (EDA) using `pandas`, store and retrieve data using `SQLite3`, and visualize patterns using the `seaborn` library.

The dataset used in this analysis contains nutritional information for popular McDonald's menu items and was obtained from Kaggle. McDonald's publishes nutritional facts to help consumers make informed dietary choices.

6 Loading and Storing the Dataset

The following Python code demonstrates how to load a CSV file into a pandas DataFrame and store it in an SQLite database.

- The `pandas` library is used to handle tabular data.
- The `sqlite3` library is used to create and connect to an SQLite database.
- The CSV file containing McDonald's nutritional data is read into a DataFrame.
- A connection to the SQLite database `mcdonalds.db` is established.
- The DataFrame is written to the database as a table named `mcdonalds_nutrition`.

If the table already exists, it is replaced. The index of the DataFrame is not stored as a database column.

```
import pandas as pd
import sqlite3

data = pd.read_csv("mcdonalds.csv")
conn = sqlite3.connect("mcdonalds.db")

data.to_sql("mcdonalds_nutrition", conn, if_exists="replace", index=False)
```

The dataset is now stored as a table named `mcdonalds_nutrition` and can be queried using SQL.

Retrieving Data from the Database

To retrieve data from the SQLite database into pandas, the `read_sql` method is used.

```
df = pd.read_sql("SELECT * FROM mcdonalds_nutrition", conn)
df.head()
```

This loads the database table into a DataFrame for analysis.

Exploratory Data Analysis with Pandas

The `describe()` method is used to generate descriptive statistics that summarize the distribution of numerical data in a DataFrame. It provides a quick overview of the dataset by computing common statistical measures for each numeric column, such as:

- `count` – number of non-null observations
- `mean` – average value
- `std` – standard deviation, measuring data dispersion
- `min` – minimum value
- `25%` – first quartile (25th percentile)
- `50%` – median (50th percentile)
- `75%` – third quartile (75th percentile)
- `max` – maximum value

This method is commonly used during exploratory data analysis (EDA) to quickly detect patterns, ranges, outliers, and potential data quality issues.

```
df.describe()
```

Analyzing Sodium Content

Sodium intake is an important dietary consideration. Using the dataset, we identify food items with high sodium content.

Summary Statistics for Sodium

```
df["Sodium"].describe()
```

The maximum sodium value is 3600 milligrams.

Identifying the Item with Maximum Sodium

```
index_max = df["Sodium"].idxmax()
df.at[index_max, "Item"]
```

The food item with the highest sodium content is *Chicken McNuggets (40 pieces)*.

Visualizing Sodium with Seaborn

A categorical scatter plot helps visualize sodium levels across food categories.

```
import seaborn as sns
sns.swarmplot(x="Category", y="Sodium", data=df)
```

The plot highlights extreme sodium values and variation between food categories.

Relationship Between Protein and Total Fat

Scatter plots are useful for analyzing relationships between numerical variables. Using a joint plot, we examine protein versus total fat.

```
sns.jointplot(x="Protein", y="Total Fat", data=df)
```

The plot shows a strong positive correlation (approximately 0.81), indicating that food items higher in protein tend to also be higher in fat. A visible outlier suggests an item with unusually high values.

Box Plot Analysis for Sugar Content

Box plots help identify data distribution and outliers.

```
sns.boxplot(y="Sugars", data=df)
```

The plot shows:

- Median sugar values around 30 grams
- Several outliers with sugar levels exceeding 100 grams

These extreme values may correspond to dessert or candy items.

Summary

In this section, we learned how to:

- Store CSV data in an SQLite database using Python
- Retrieve SQL data into pandas DataFrames
- Perform exploratory data analysis using pandas
- Visualize patterns and outliers using seaborn

Combining databases, Python, and visualization tools enables efficient and insightful data analysis.